

Sesión 10: Monitorización y cierre del ciclo

Duración: 2,5 horas | Modalidad: Virtual síncrona

Objetivo	Entender que el trabajo no termina con el despliegue. Detectar data drift y concept drift con Evidently AI y cerrar el ciclo MLOps completo.
Herramientas	Evidently AI, scikit-learn, pandas, numpy, Jupyter, FastAPI (Prometheus opcional)
Prerrequisitos	Stack de la sesión 9 funcionando. pip install evidently pandas scikit-learn

BLOQUE 1 — Monitorización DevOps del servicio (20 min)

Antes de hablar de monitorización ML, el servicio necesita la monitorización básica de cualquier aplicación web. Las métricas fundamentales son latencia, tasa de errores y uptime.

Añadir métricas Prometheus a FastAPI

```
# pip install prometheus-fastapi-instrumentator

from fastapi import FastAPI
from prometheus_fastapi_instrumentator import Instrumentator

app = FastAPI()

# Registra automáticamente métricas de todas las rutas:
# - http_requests_total (contador por endpoint y status code)
# - http_request_duration_seconds (histograma de latencias)
Instrumentator().instrument(app).expose(app)

# Ahora en http://localhost:8000/metrics tienes las métricas
# en formato Prometheus para que Grafana las visualice
```

Para este curso

No configuramos Prometheus + Grafana completo (requeriría otra sesión). El objetivo es saber que existe, entender qué métricas hay que monitorizar y saber cómo exponerlas. El foco de hoy es la monitorización específica de ML.

BLOQUE 2 — Data Drift y Concept Drift: teoría (20 min)

Esta es la parte de monitorización exclusiva de MLOps que no existe en DevOps tradicional.

Data Drift	Concept Drift
La distribución de los datos de entrada ha cambiado respecto al entrenamiento.	La relación entre las features y el target ha cambiado. El mundo ha cambiado.
Ej: el vocabulario del texto cambió (slang nuevo, nuevos productos)	Ej: palabras que antes eran positivas ahora son irónicas o negativas
Se detecta comparando distribuciones de features	Se detecta monitorizando métricas del modelo en producción

BLOQUE 3 — Evidently AI: informe de drift en la práctica (40 min)

Evidently AI genera informes HTML interactivos que comparan dos datasets: los datos de referencia (entrenamiento) y los datos actuales (producción simulada).

Instalación:

```
pip install evidently pandas scikit-learn numpy
```

Fichero: generate_drift_report.py

```
import pandas as pd
import numpy as np
from datasets import load_dataset
from transformers import pipeline
from evidently.report import Report
from evidently.metric_preset import DataDriftPreset, TextOverviewPreset
from evidently.metrics import (
    DatasetDriftMetric,
    ColumnDriftMetric,
    DatasetMissingValuesMetric
)
import warnings
warnings.filterwarnings('ignore')

# — 1. PREPARAR LOS DATOS —————
print('Cargando datos...')

# Datos de referencia: validación estándar del SST-2
ref_dataset = load_dataset('glue', 'sst2', split='validation[:300]')

# Datos 'de producción' simulados: usamos el split de test
# En la realidad serían los textos que llegan en producción
```

```

prod_dataset = load_dataset('glue', 'sst2', split='test[:300]')

# — 2. EXTRAER FEATURES PARA EL ANÁLISIS —————
# Evidently trabaja con DataFrames. Extraemos features del texto.

def extract_features(dataset):
    texts = dataset['sentence']
    labels = dataset['label']

    # Features textuales simples
    df = pd.DataFrame({
        'text': texts,
        'label': labels,
        'text_length': [len(t) for t in texts],
        'word_count': [len(t.split()) for t in texts],
        'avg_word_len': [np.mean([len(w) for w in t.split()]) for t in texts],
        'excl_count': [t.count('!') for t in texts],
        'question_count': [t.count('?') for t in texts],
        'upper_ratio': [sum(1 for c in t if c.isupper()) / max(len(t), 1) for t
in texts]
    })
    return df

print('Extrayendo features...')
reference_df = extract_features(ref_dataset)
production_df = extract_features(prod_dataset)

# — 3. AÑADIR PREDICCIONES DEL MODELO —————
# Para un análisis completo, añadimos las predicciones del modelo
print('Generando predicciones del modelo...')
clf = pipeline('sentiment-analysis',
                model='distilbert-base-uncased-finetuned-sst-2-english',
                truncation=True, max_length=128)

label_map = {'POSITIVE': 1, 'NEGATIVE': 0}

def add_predictions(df):
    results = clf(df['text'].tolist(), batch_size=32)
    df['prediction'] = [label_map[r['label']] for r in results]
    df['confidence'] = [r['score'] for r in results]
    df['correct'] = (df['prediction'] == df['label']).astype(int)
    return df

reference_df = add_predictions(reference_df)
production_df = add_predictions(production_df)

print(f'Referencia → accuracy: {reference_df["correct"].mean():.3f}')
print(f'Producción → accuracy: {production_df["correct"].mean():.3f}')

# — 4. GENERAR INFORME DE DRIFT —————
print('Generando informe de drift...')

# Informe completo con preset de drift + métricas individuales
report = Report(metrics=[

```

```

    DatasetDriftMetric(),                # ¿Ha derivado el dataset en general?
    DatasetMissingValuesMetric(),        # ¿Hay más valores nulos?
    ColumnDriftMetric(column_name='text_length'),
    ColumnDriftMetric(column_name='word_count'),
    ColumnDriftMetric(column_name='confidence'),
    ColumnDriftMetric(column_name='correct'), # Drift en la calidad del modelo
    DataDriftPreset()                    # Análisis completo de todas las columnas
])

report.run(
    reference_data=reference_df.drop(columns=['text']),
    current_data=production_df.drop(columns=['text'])
)

# Guardar informe HTML
report.save_html('drift_report.html')
print('Informe guardado en drift_report.html')

# — 5. EXTRAER RESULTADOS PROGRAMÁTICAMENTE —————
# En un pipeline de CI/CD queremos resultados en JSON, no HTML
results = report.as_dict()

# Encontrar si hay drift significativo
dataset_drift = results['metrics'][0]['result']
print(f"\nResumen de drift:")
print(f" Columnas con drift: {dataset_drift.get('number_of_drifted_columns',
'N/A')}")
print(f" Proporción de drift: {dataset_drift.get('share_of_drifted_columns',
'N/A'):.1%}")
print(f" Dataset drifted: {dataset_drift.get('dataset_drift', 'N/A')}")

```

Fichero: monitor_production.py — alertas automáticas

```

# Script que ejecutar periódicamente (cron job, GitHub Actions programado)
# Compara el rendimiento actual con el de referencia y alerta si hay drift

import json
import sys
import pandas as pd
from evidently.report import Report
from evidently.metrics import DatasetDriftMetric, ColumnDriftMetric

DRIFT_THRESHOLD = 0.2 # Alerta si > 20% de columnas con drift
ACCURACY_DROP_THRESHOLD = 0.05 # Alerta si accuracy cae > 5 puntos

def check_drift(reference_df: pd.DataFrame, production_df: pd.DataFrame) -> dict:
    report = Report(metrics=[
        DatasetDriftMetric(),
        ColumnDriftMetric(column_name='correct') # Métrica clave del modelo
    ])
    report.run(reference_data=reference_df, current_data=production_df)
    results = report.as_dict()

```

```

drift_result = results['metrics'][0]['result']
quality_result = results['metrics'][1]['result']

ref_accuracy = reference_df['correct'].mean()
prod_accuracy = production_df['correct'].mean()
accuracy_drop = ref_accuracy - prod_accuracy

alerts = []
if drift_result.get('share_of_drifted_columns', 0) > DRIFT_THRESHOLD:
    alerts.append(f'DATA DRIFT: {drift_result["share_of_drifted_columns"]:.1%}
de columnas con drift')
    if accuracy_drop > ACCURACY_DROP_THRESHOLD:
        alerts.append(f'ACCURACY DROP: bajó {accuracy_drop:.1%}
(ref={ref_accuracy:.3f}, prod={prod_accuracy:.3f})')

return {
    'alerts': alerts,
    'drift_share': drift_result.get('share_of_drifted_columns', 0),
    'accuracy_ref': round(ref_accuracy, 4),
    'accuracy_prod': round(prod_accuracy, 4),
    'accuracy_drop': round(accuracy_drop, 4),
    'needs_retraining': len(alerts) > 0
}

# En producción, cargarías estos DataFrames desde tu almacén de datos
# Aquí simulamos con datos aleatorios para demostrar la lógica
if __name__ == '__main__':
    # Simular datos: referencia sana vs producción con drift
    np.random.seed(42)
    ref_df = pd.DataFrame({'text_length': np.random.normal(80, 20, 500),
                           'word_count': np.random.normal(15, 5, 500),
                           'correct': np.random.binomial(1, 0.92, 500)})
    # Producción simulada con drift: textos más largos y peor rendimiento
    prod_df = pd.DataFrame({'text_length': np.random.normal(120, 40, 500), #
DRIFT
                           'word_count': np.random.normal(22, 8, 500), #
DRIFT
                           'correct': np.random.binomial(1, 0.85, 500)}) #
DROP

    result = check_drift(ref_df, prod_df)
    print(json.dumps(result, indent=2))

    if result['needs_retraining']:
        print('\nACCION REQUERIDA: Re-entrenamiento recomendado')
        for alert in result['alerts']:
            print(f' - {alert}')
        sys.exit(1) # Código de salida no-cero para fallar en CI/CD
    else:
        print('\nModelo en buen estado. No se requiere acción.')
        sys.exit(0)

```

Revisión visual de todo lo que hemos construido en 10 semanas:

Semana	Herramienta	Problema que resuelve
1-2	Git	Versionar el código y colaborar sin conflictos
3	DVC	Versionar datos y modelos (lo que Git no puede)
4	Docker	'En mi máquina funciona' — nunca más
5	GitHub Actions	Automatizar CI/CD: lint, tests, build, deploy
6	MLflow Tracking	Registrar experimentos de forma sistemática
7	MLflow Registry	Catálogo oficial de modelos con ciclo de vida
8	FastAPI	Exponer el modelo como servicio REST consumible
9	Docker Compose	Orquestar el stack completo en producción
10	Evidently AI	Detectar cuándo el modelo empieza a fallar

El ciclo completo que cierra hoy

